Keine's Classroom Battle

Kirk Lester Mangco
College of Computer and Information Sciences
Mapua Malayan Colleges Mindanao
Davao City, Philippines
klMangco@mcm.edu.ph

Abstract—**This paper presents a new yet familiar way for students to engage in schoolwork when their attention or lack of motivation gets in the way of learning. Keine's Classroom Battle is a game that lets students quiz themselves and test their knowledge or compete with their peers in a classroom setting. This paper explains the process of the game's creation. Despite the use of Godot, the project incorporates Object-Oriented Programming (OOP) principles such as inheritance, encapsulation, and abstraction. The paper also presents how the structure mimics the MVC (Model-View-Architecture) framework within Godot's setting. The application created serves as a functional tool that both educators and students can utilize, aligning with the United Nations Sustainable Development Goal (SDG) 4: Quality Education**

I. Introduction

In todays classrooms, students often struggle with keeping attention to their lectures, while on the other hand, teachers struggle to retain learner attention. By applying the underlying foundation of how educational games like 'Kahoot!' can excite a classroom full of learners to score high points, Keine hopes to utilize classroom competition as a means to invigorate learners and assist teachers through classroom activities and self-study sessions. Keine is a multiplayer party quiz game that lets users upload custom JSON files containing questions, options, and answers. For those who seek to use the game as a self-study tool, the singleplayer mode exists which allows the use to add difficulty levels ranging from 1 to 3 with categories so that specific statistics can be gathered at the end of the quiz. It is through this that Keine directly supports the SDG 4 (Quality Education).

II. Methods

The game application was created using the Godot game engine for its beginner-friendly approach to creating games, utilizing the concept of OOP and MVC.

A. User Journey

1. Player opens the game
2. Player selects either Singleplayer or Multiplayer
3. Once in Multiplayer, the user can either host a lobby or join a friend's lobby via the available lobbies or a friend invite
    a. The host selects the JSON file
    b. The host can set the timer used for the quiz section (defaults at 30 seconds otherwise)
    c. The host then starts the game and the phase shifts from LOBBY to QUESTION
    d. The users select answers from 4 options given
    e. After each question, the results show which users got it correct and who has the highest points based on correct answer and time remaining.
    f. The host then clicks the "Next Round" button to proceed to the next question.
    g. After the quiz ends, the players will return to the lobby where they can either disconnect or do another
4. Once in Singleplayer, the player will enter an offline instance of the game lobby.
    a. The player chooses the JSON file that contains the quiz data (with difficulties and subjects/categories).
    b. The player can set the timer used for the quiz section (defaults at 30 seconds otherwise)
    c. The player will go through the quiz in a randomized order, going from level 1 (easy) to level 3 (hard).
    d. The user will select an answer out of 4 options.
    e. After the timer ends, the screen will display the correct answer, then transition to the next question.
    f. Once complete, the results will show which subject/s the player found hard, displaying the number of questions the player answered correctly.
    g. The player then returns to the Singleplayer lobby, where they can change their quiz or timer length.

B. Class Diagram

The class diagram model showcases the game functions of Keine. Instead of actual classes being used, nodes and scripts were utilized as they serve the same function.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

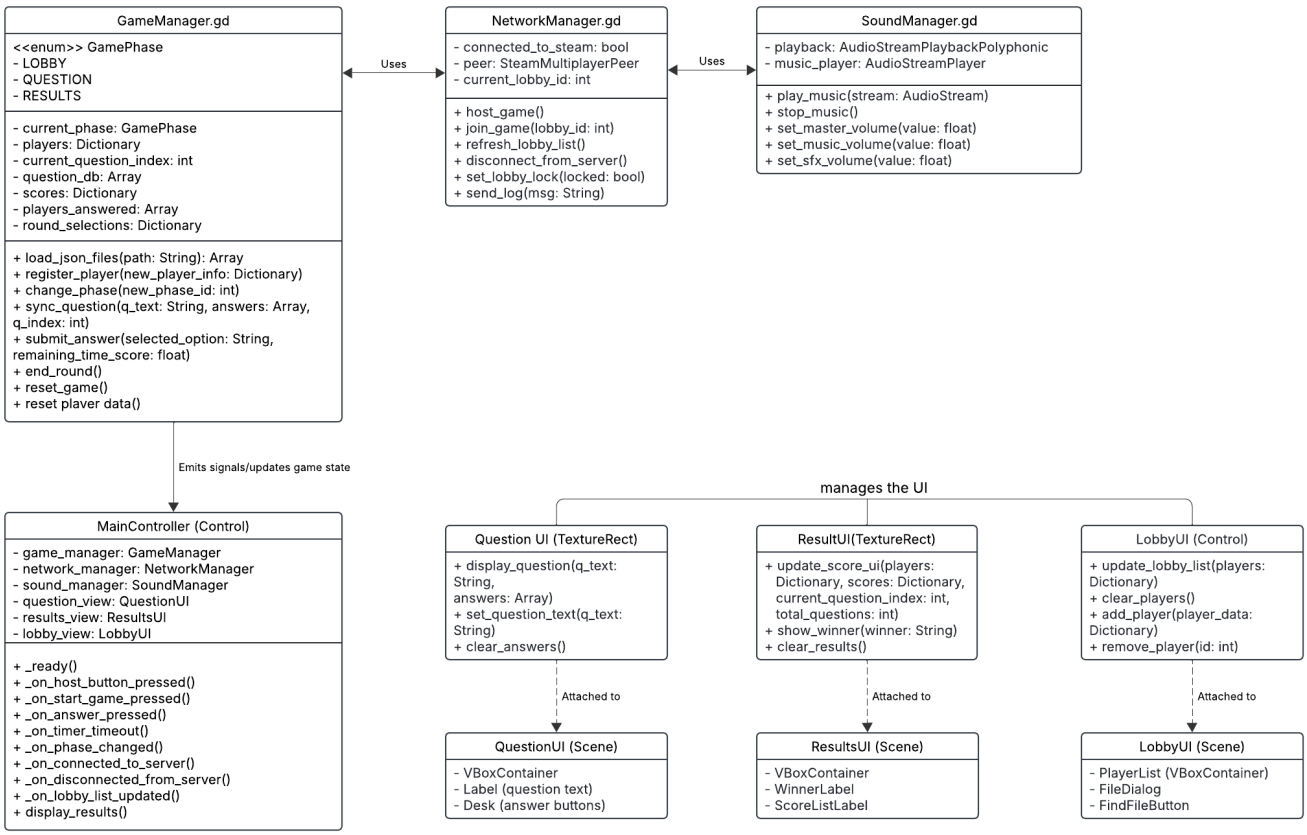


Fig. 1. Class Diagram of the Multiplayer Game Loop

C. *Frameworks And Tools Used*

- Godot – It is a free and open-source game engine used to develop the game. It uses GDScript whose syntax is like that of Python. The engine also uses a node-and-scene architecture, meaning that every screen is comprised of scenes with interactions assigned to nodes (such as buttons, input fields, and scoll bars). This architecture allows for easy reusability between scenes. Utilizing Autoloads/singletons, game logic can be relegated to scripts away from the root nodes of a scene.

- Krita – It is a free and open-source raster graphics editor that is primarily designed for digital artists and 2D animators. This is the software that was used to create the 2D assets present in the game.

D. *OOP Principles*

The concept of concept of OOP was utilized during the development of the game. Taking advantage of Godot's node structures and native classes, logic and data have been separated between scripts and scenes, making sure that the user is only responsible for interacting and responding to the UI.

- Encapsulation
  This allows for the bundling of related data and methods that operate on the data together, controlling how the data is accessed or modified. Since GameManager.gd is a singleton, it can be accessed globally, meaning this only counts as partial encapsulation.

```
var players = {}
var current_question_index = 0
var question_db = []
var scores = {} #Format: {peer_id, score}
var players_answered = []
var round_selections = {} #Format: {peer_id: selected_option}
```

Fig. 2. Screenshot of GameManager.gd showing the grouped data

```
#any_peer allows clients to call this on the server
@rpc("any_peer", "call_local", "reliable")
func submit_answer(selected_option, remaining_time_score):
    #for security, only the server should access the score
    if multiplayer.is_server():
        var player_id = multiplayer.get_remote_sender_id()

        #prevent duplicates if they spam the button somehow
        if player_id in players_answered:
            return

        #mark the player as answered
        players_answered.append(player_id)
        round_selections[player_id] = selected_option

        var correct_option = question_db[current_question_index]["answer"]
        print("SERVER CHECK - User selected: '", str(selected_option), "' | Correct JSON answer: '", str(correct_option), "'")

        if str(selected_option) == correct_option:
            NetworkManager.send_log("Player " + str(player_id) + " got it right!")
            print("+1")
            if not scores.has(player_id):
                scores[player_id] = 0.0
            scores[player_id] += (1.0 + remaining_time_score/1000.0)


        #wait for everyone to answer
        if players_answered.size() == players.size():
            end_round()
```

Fig. 3. Screenshot of submit_answer showing how it processes the answer and scores

- Abstraction
  This allows for the hiding of complicated

implementation details behind simple methods that other parts of the program can use.

```
func host_game():
    if not connected_to_steam or not peer:
        send_log("Cannot host: Steam is not active.")
        return

    send_log("Attempting to host...")
    peer.create_host(0)
    multiplayer.multiplayer_peer = peer
    Steam.createLobby(Steam.LOBBY_TYPE_PUBLIC, 50)
```

Fig. 4. Screenshot of host_game inside NetworkManager, allowing for the function to be called anywhere in the project.

- Inheritance
  This allows for classes (or nodes) to inherit properties from its parent class. For example, by since the UI script extends Control, it can use the UI-related functionality such as layout behavior



Fig. 5. Screenshot of TextureRect inheriting node, therefore inheriting the script and attributes Node has.

- Polymorphism
  While not explicitly used within the game's code, Godot creates functions such as _ready() and _process() when creating new scripts. The former can have subclasses themselves differently, while the latter can have subclasses perform different updates per-frame.

III. Results and Discussions

Despite the initial bugs and issues during development, the game was praised by the testers. Bugs were encountered especially during the multiplayer testing of the project as the game utilizes Steam's multiplayer test server, resulting in playtesters unable to find friend created lobbies, though the testers were still able to invite each other through Steam invites. From our testing, this is due to Steam's regional differences. Despite setting the lobby visibility to worldwide, the browser simply returns the nearest lobbies within the region that also use the test server.

IV. Conclusion

Keine's Classroom was developed successfully, albeit with issues still present. However, for a basic quiz game, it still fulfills it purpose appropriately.

The first issue to tackle would be to get a Steam Developer account, as this allows for a smoother multiplayer experience without the need for test servers, circumventing the issue with players unable to find their friend's lobby. More polish is needed especially within the code itself, which would be the second priority to ensure efficient and smooth performance. Assets can be replaced when time is available, and adding more customizability within the options menu.